

SDSE Comprehensive Notes

Software Design & Software Engineering

Complete Exam Preparation Guide

Covers: OOP · UML · ERD · Design Patterns · SOLID · SDLC · Testing

Exam Format: 60% MCQs + 40% Coding

Focus on: definitions, comparisons, code identification, and practical application

Contents

1	1. Software Engineering	3
1.1	1.1 Software Lifecycle	3
1.2	1.2 Quality Attributes	3
2	2. Object-Oriented Programming (OOP)	3
2.1	2.1 Core Concepts	3
2.1.1	2.1.1 Classes & Objects	4
2.1.2	2.1.2 Constructors	4
2.1.3	2.1.3 Invariants	4
2.2	2.2 OOP Mechanisms	5
2.2.1	2.2.1 Encapsulation	5
2.2.2	2.2.2 Abstraction	5
2.2.3	2.2.3 Inheritance	6
2.2.4	2.2.4 Polymorphism	7
2.3	2.3 Object Relationships & Design Concepts	7
2.3.1	2.3.1 Composition	7
2.3.2	2.3.2 DTO / Domain Object	7
2.3.3	2.3.3 Observability	8
3	3. Unified Modelling Language (UML)	8
3.1	3.1 Structural UML Diagrams	8
3.1.1	3.1.1 Class Diagram	8
3.1.2	3.1.2 Class Relationships	8
3.2	3.2 Behavioural UML Diagrams	8
3.2.1	3.2.1 Sequence Diagram	8
3.2.2	3.2.2 Use Case Diagram	9
4	4. Entity Relationship Diagram (ERD)	9
4.1	4.1 ERD Components	9
4.2	4.2 Mapping Cardinalities	9
4.3	4.3 Participation Constraints	9
4.4	4.4 ERD Notations	9
4.4.1	4.4.1 Crow's Foot Notation	10
4.4.2	4.4.2 Chen Notation	10
5	5. Design Patterns	10
5.1	5.1 Creational Patterns	10
5.1.1	5.1.1 Singleton Pattern	10
5.1.2	5.1.2 Factory Pattern	11
5.2	5.2 Structural Patterns	11
5.2.1	5.2.1 Adapter Pattern	11
5.2.2	5.2.2 Composite Pattern	12
5.3	5.3 Behavioural Patterns	12
5.3.1	5.3.1 Observer Pattern	12
5.3.2	5.3.2 Strategy Pattern	13
6	6. SOLID Principles	13
6.1	6.1 S — Single Responsibility Principle (SRP)	13
6.2	6.2 O — Open/Closed Principle (OCP)	14
6.3	6.3 L — Liskov Substitution Principle (LSP)	14
6.4	6.4 I — Interface Segregation Principle (ISP)	14

6.5	D — Dependency Inversion Principle (DIP)	15
7	7. Software Development Life Cycle (SDLC)	15
7.1	7.1 SDLC Overview	15
7.2	7.2 SDLC Models	15
7.2.1	7.2.1 Waterfall Model	16
7.2.2	7.2.2 Agile Model	16
8	8. Testing	16
8.1	8.1 What is Testing?	17
8.2	8.2 Types of Testing	17
8.2.1	8.2.1 Testing by Scope	17
8.2.2	8.2.2 Box Testing	17
8.2.3	8.2.3 Manual vs Automated Testing	17
8.2.4	8.2.4 Functional vs Non-Functional Testing	18
	Quick Reference Cheat Sheet	18

1. Software Engineering

1.1 Software Lifecycle

Definition

A **Software Lifecycle** is the structured process of planning, creating, testing, and delivering a software system. It defines *what* should be done at each stage and *how* different phases relate to each other.

Core Phases (common to all models):

1. **Requirements** — gather and document what the system must do
2. **System Design** — architecture, database schema, tech choices
3. **Implementation (Coding)** — write the actual code
4. **Testing** — verify correctness and quality
5. **Deployment** — release to production
6. **Maintenance** — bug fixes, enhancements post-release

Key Point

The lifecycle provides **structure**, improves **predictability**, reduces **rework**, and ensures **quality**. Without it, projects are chaotic and hard to manage.

1.2 Quality Attributes

Quality attributes (also called **non-functional requirements**) define *how well* a system performs its functions.

Attribute	Description	MCQ Trigger Word
Reliability	System works correctly without failure over time	“uptime”, “fault”
Maintainability	Ease of modifying, fixing, or extending code	“change”, “update”
Scalability	Handles increased load without degradation	“users grow”, “load”
Performance	Speed, response time, throughput	“fast”, “latency”
Security	Protection from unauthorised access or attacks	“breach”, “auth”
Usability	How easy it is for users to interact with the system	“UX”, “user-friendly”
Portability	Runs on different platforms / environments	“cross-platform”
Testability	How easy it is to write and run tests	“test coverage”

Exam Tip

MCQs often ask you to **identify which quality attribute** is being described. Match the scenario to the attribute using the trigger words above.

2. Object-Oriented Programming (OOP)

2.1 Core Concepts

Classes & Objects

Class vs Object

- **Class** — a blueprint / template that defines fields and methods
- **Object** — a concrete *instance* of a class, with its own state

```
1 class Car {
2     String brand;    // field
3     int speed;
4
5     void accelerate() { speed += 10; } // method
6 }
7
8 Car myCar = new Car(); // myCar is an object (instance)
9 myCar.brand = "Toyota";
```

Constructors

Definition

A **constructor** is a special method called automatically when an object is created. It has the same name as the class and no return type.

```
1 class Car {
2     String brand;
3     int speed;
4
5     // Default constructor
6     Car() { this.brand = "Unknown"; this.speed = 0; }
7
8     // Parameterised constructor
9     Car(String brand, int speed) {
10         this.brand = brand;
11         this.speed = speed;
12     }
13 }
14 Car c1 = new Car();           // calls default constructor
15 Car c2 = new Car("BMW", 200); // calls parameterised constructor
```

Key Point

Types: Default (no args), Parameterised, Copy constructor. If you define *any* constructor, Java no longer provides the default one.

Invariants

Definition

An **invariant** is a condition or rule that must *always be true* for an object throughout its lifetime (before and after every method call).

Example: A `BankAccount` invariant: $\text{balance} \geq 0$. The class must ensure no operation can leave balance negative.

```
1 class BankAccount {
2     private double balance;
3     BankAccount(double initial) {
4         if (initial < 0) throw new IllegalArgumentException("Negative balance");
5         this.balance = initial; // invariant established
6     }
7     void withdraw(double amount) {
8         if (amount > balance) throw new IllegalStateException("Insufficient funds");
```

```

9      balance -= amount;  // invariant maintained
10  }
11 }

```

2.2 OOP Mechanisms

Encapsulation

Definition

Encapsulation is the bundling of data (fields) and methods that operate on that data within a single unit (class), and **restricting direct access** to some components using access modifiers.

Access Modifiers:

Modifier	Same Class	Same Package	Subclass	Anywhere
private	✓	×	×	×
(default)	✓	✓	×	×
protected	✓	✓	✓	×
public	✓	✓	✓	✓

Getters and Setters:

```

1  class Person {
2      private String name;  // hidden data
3      private int age;
4
5      // Getter
6      public String getName() { return name; }
7
8      // Setter with validation
9      public void setAge(int age) {
10         if (age < 0) throw new IllegalArgumentException("Age cannot be negative");
11         this.age = age;
12     }
13 }

```

Key Point

Encapsulation = **data hiding**. Benefits: control over data, easier to change implementation without breaking consumers.

Abstraction

Definition

Abstraction means hiding the *implementation details* and showing only the *essential features* (the “what”, not the “how”).

Abstract Class:

```

1  abstract class Shape {
2      abstract double area();  // no implementation      subclass must provide
3      void display() { System.out.println("Area: " + area()); } // concrete method
4  }
5
6  class Circle extends Shape {
7      double radius;
8      Circle(double r) { this.radius = r; }
9      @Override
10     double area() { return Math.PI * radius * radius; }
11 }

```

Key Point

- **Abstract class:** can have both abstract and concrete methods; *cannot* be instantiated; use `extends`
- **Interface:** all methods are abstract by default (Java 8+: can have `default` methods); use `implements`; supports multiple inheritance

Interface Example:

```

1 interface Drawable {
2     void draw();           // abstract by default
3     default void resize() { System.out.println("Resizing..."); } // default method
4 }
5
6 class Circle implements Drawable {
7     public void draw() { System.out.println("Drawing circle"); }
8 }

```

Abstract Class vs Interface — Quick Comparison:

Feature	Abstract Class	Interface
Instantiation	No	No
Concrete methods	Yes	Only default/static (Java 8+)
Fields	Any	Only public static final
Multiple inheritance	No	Yes
Constructor	Yes	No
Keyword	<code>extends</code>	<code>implements</code>

Contracts: An interface acts as a **contract** — any class implementing it *must* provide all the declared methods. This guarantees a consistent API regardless of internal implementation.

Inheritance**Definition**

Inheritance allows a child (sub) class to *acquire* fields and methods of a parent (super) class using the `extends` keyword. It models an **IS-A** relationship.

```

1 class Animal {
2     String name;
3     void eat() { System.out.println(name + " is eating"); }
4 }
5
6 class Dog extends Animal {
7     void bark() { System.out.println(name + " is barking"); }
8 }
9
10 Dog d = new Dog();
11 d.name = "Rex";
12 d.eat(); // inherited from Animal
13 d.bark(); // Dog's own method

```

Key Point

Use `super` to call the parent constructor or method. Java supports **single inheritance** for classes (but multiple via interfaces).

Polymorphism

Definition

Polymorphism means “many forms”. An object can take multiple forms depending on context. The same method name can behave differently based on the object.

Two types:

1. **Compile-time (Method Overloading)** — same method name, different parameters:

```
1 class Calc {
2     int add(int a, int b) { return a + b; }
3     double add(double a, double b) { return a + b; }
4 }
```

2. **Runtime (Method Overriding)** — subclass redefines parent method:

```
1 Animal a = new Dog(); // polymorphic reference
2 a.sound(); // calls Dog's sound(), not Animal's -> runtime decision
```

Exam Tip

Overloading vs Overriding:

- Overloading = same class, same name, different signature → compile-time
- Overriding = parent-child, same signature → runtime, needs @Override

2.3 Object Relationships & Design Concepts

Composition

Definition

Composition is a **HAS-A** relationship where one class contains another as a field, and the contained object's lifetime is *tied* to the container (strong ownership).

```
1 class Engine {
2     void start() { System.out.println("Engine started"); }
3 }
4
5 class Car {
6     private Engine engine = new Engine(); // Car HAS-A Engine
7     void drive() { engine.start(); }
8 }
9 // If Car is destroyed, Engine is also destroyed
```

Key Point

Composition over Inheritance is a design principle — prefer composing behaviours rather than inheriting them, for greater flexibility.

Aggregation (weaker form): HAS-A but the child can exist independently (e.g., University HAS-A Student).

DTO / Domain Object

	DTO (Data Transfer Object)	Domain Object
Purpose	Transfer data between layers	Represent business logic/entity
Methods	Only getters/setters	Business methods
Behaviour	None	Has behaviour
Example	UserDTO for API response	User with login(), resetPassword()

Observability

Definition

Observability is the ability to understand the internal state of a system from its external outputs. The three pillars are:

- **Logs** — discrete timestamped events (“User logged in at 10:05”)
- **Metrics** — numeric measurements over time (CPU usage, request count)
- **Traces** — track a request’s path through distributed services

3. Unified Modelling Language (UML)

3.1 Structural UML Diagrams

Class Diagram

A **Class Diagram** shows classes, their attributes, methods, and the relationships between them.

Class box structure:

ClassName
- privateField: Type + publicField: Type
+ publicMethod(): ReturnType - privateMethod(param: Type): void

Visibility symbols:

- + public - private # protected ~ package

Class Relationships

Relationship	Notation	Meaning
Association	Solid line	General “uses” relationship
Aggregation	Hollow diamond	HAS-A (weak), child survives independently
Composition	Filled diamond	HAS-A (strong), child dies with parent
Inheritance	Hollow arrow	IS-A, subclass extends superclass
Realisation	Dashed + hollow arrow	Class implements interface
Dependency	Dashed arrow	One class uses another temporarily

Exam Tip

Know the diamond direction: diamond is on the **owner** side.
Hollow diamond = Aggregation; Filled diamond = Composition.

3.2 Behavioural UML Diagrams

Sequence Diagram

Shows **how objects interact over time**, step by step, in a specific scenario.

Key elements:

- **Lifelines** — vertical dashed lines representing participants
- **Activation bars** — rectangles on lifelines showing active periods
- **Messages** — horizontal arrows between lifelines (solid = call, dashed = return)
- **Self-call** — arrow looping back to the same lifeline

Reading order: Top to bottom = chronological order of events.

Use Case Diagram

Shows **functional requirements** from the user's perspective.

Elements:

- **Actor** — stick figure; a user or external system that interacts with the system
- **Use Case** — oval/ellipse; a system function or feature
- **System boundary** — rectangle enclosing all use cases
- **<<include>>** — one use case *always* triggers another (mandatory)
- **<<extend>>** — one use case *optionally* extends another (conditional)

Exam Tip

include = always happens (e.g., Login always checks credentials).
extend = sometimes happens (e.g., Checkout *may* apply coupon).

4. Entity Relationship Diagram (ERD)

4.1 ERD Components

Definition

An **ERD** is a visual representation of the data model showing entities, their attributes, and how they relate to each other.

- **Entity** — a real-world object/concept with data stored (rectangle). E.g., **Student**, **Course**
- **Weak Entity** — an entity that cannot be uniquely identified without a related entity (double rectangle). E.g., **OrderItem** depends on **Order**
- **Attribute** — a property of an entity (ellipse):
 - **Simple** — single atomic value (Age)
 - **Composite** — can be split further (FullName → First + Last)
 - **Derived** — computed from another attribute (*dashed* ellipse, e.g., Age from DOB)
 - **Multi-valued** — can have multiple values (*double* ellipse, e.g., PhoneNumbers)
 - **Primary Key** — uniquely identifies an entity (underlined)
- **Relationship** — association between entities (diamond shape)

4.2 Mapping Cardinalities

Cardinality defines how many instances of one entity relate to instances of another.

Type	Meaning	Real-world Example
One-to-One (1:1)	One entity relates to exactly one other	Person — Passport
One-to-Many (1:N)	One entity relates to many others	Teacher — Students
Many-to-One (N:1)	Many entities relate to one	Orders — Customer
Many-to-Many (M:N)	Many relate to many (needs a join table)	Students — Courses

4.3 Participation Constraints

- **Total Participation** — every entity *must* participate in the relationship (double line). E.g., every Employee *must* work in a Department.
- **Partial Participation** — entity *may or may not* participate (single line). E.g., some Employees *may* manage a Department.

4.4 ERD Notations

Crow's Foot Notation

Used in database tools (Lucidchart, MySQL Workbench).

Symbol at line end	Meaning
(single line)	Exactly one
o (circle)	Zero
< (crow's foot)	Many
(double line)	Mandatory

Common combinations: o| = zero or one o< = zero or many || = one and only one |< = one or many

Chen Notation

The original ERD notation using geometric shapes:

- Rectangles = Entities Ellipses = Attributes Diamonds = Relationships
- Numbers (1, N, M) placed on relationship lines to show cardinality

5. Design Patterns

Definition

A **Design Pattern** is a reusable solution to a commonly occurring design problem. Categorised by GoF (Gang of Four) into: **Creational, Structural, Behavioural**.

5.1 Creational Patterns

Singleton Pattern

Singleton

Ensures a class has **only one instance** and provides a **global access point** to it.

Use cases: Database connection pool, Logger, Configuration manager, Thread pool.

```

1 public class Singleton {
2     private static Singleton instance = null; // only one instance
3
4     private Singleton() {} // private constructor blocks external creation
5
6     public static Singleton getInstance() {
7         if (instance == null) {
8             instance = new Singleton(); // lazy initialisation
9         }
10        return instance;
11    }
12 }
13
14 // Usage:
15 Singleton s1 = Singleton.getInstance();
16 Singleton s2 = Singleton.getInstance();
17 // s1 == s2 --> TRUE, same object!

```

Key Point

Thread-safe Singleton uses **synchronized** or **double-checked locking**: `if (instance == null) { synchronized(Singleton.class) { if (instance == null) { instance = new`

```
Singleton(); }}
```

Factory Pattern

Factory

Defines an interface for **creating objects**, but lets **subclasses/factory methods decide** which class to instantiate. Decouples object creation from usage.

Use cases: When exact class of object to create isn't known ahead of time.

```
1 interface Animal { void speak(); }
2
3 class Dog implements Animal { public void speak() { System.out.println("Woof"); } }
4 class Cat implements Animal { public void speak() { System.out.println("Meow"); } }
5
6 class AnimalFactory {
7     public static Animal create(String type) {
8         switch (type) {
9             case "dog": return new Dog();
10            case "cat": return new Cat();
11            default: throw new IllegalArgumentException("Unknown type");
12        }
13    }
14 }
15
16 // Usage      client doesn't know which concrete class is used:
17 Animal a = AnimalFactory.create("dog");
18 a.speak();    // Woof
```

Exam Tip

Factory = **centralise object creation**, decouple caller from concrete types.
MCQ trick: “Which pattern hides the instantiation logic?” → **Factory**.

5.2 Structural Patterns

Adapter Pattern

Adapter

Converts the interface of one class into another interface that clients expect. Acts as a **wrapper** to make incompatible interfaces work together.

Real-world analogy: A power plug adapter that converts US sockets to UK sockets.

```
1 // Existing (incompatible) 3rd-party class
2 class OldLogger { public void writeLog(String msg) { System.out.println(msg); } }
3
4 // Target interface our system expects
5 interface Logger { void log(String msg); }
6
7 // Adapter wraps the old class and provides the new interface
8 class LoggerAdapter implements Logger {
9     private OldLogger old = new OldLogger();
10    public void log(String msg) { old.writeLog(msg); } // adapts the call
11 }
12
13 // Now our system can use OldLogger through the Logger interface
14 Logger logger = new LoggerAdapter();
15 logger.log("System started");
```

Composite Pattern

Composite

Composes objects into **tree structures** to represent part-whole hierarchies. Lets clients treat individual objects and groups of objects **uniformly**.

Real-world example: File system — a Folder can contain Files or other Folders. Both share the same FileSystemItem interface.

```
1 interface FileSystemItem { void display(int indent); }
2
3 class File implements FileSystemItem {
4     String name;
5     public void display(int i) { System.out.println(" ".repeat(i) + name); }
6 }
7
8 class Folder implements FileSystemItem {
9     String name;
10    List<FileSystemItem> children = new ArrayList<>();
11    public void add(FileSystemItem item) { children.add(item); }
12    public void display(int i) {
13        System.out.println(" ".repeat(i) + "[" + name + "]");
14        for (FileSystemItem child : children) child.display(i + 2);
15    }
16 }
```

5.3 Behavioural Patterns

Observer Pattern

Observer

Defines a **one-to-many dependency** between objects so that when one object (Subject/Publisher) changes state, all its dependents (Observers/Subscribers) are notified and updated automatically.

Use cases: Event listeners, GUI frameworks, MVC (Model notifies View), notifications.

```
1 interface Observer { void update(String event); }
2
3 class EventSystem {
4     private List<Observer> observers = new ArrayList<>();
5     public void subscribe(Observer o) { observers.add(o); }
6     public void publish(String event) {
7         for (Observer o : observers) o.update(event); // notify all
8     }
9 }
10
11 class EmailService implements Observer {
12     public void update(String event) { System.out.println("Email sent: " + event); }
13 }
14
15 // Usage:
16 EventSystem es = new EventSystem();
17 es.subscribe(new EmailService());
18 es.publish("Order placed"); // EmailService notified automatically
```

Strategy Pattern

Strategy

Defines a family of algorithms, encapsulates each one, and makes them **interchangeable at runtime**. The strategy can be swapped without changing the client code.

```

1 interface SortStrategy { void sort(int[] data); }
2
3 class BubbleSort implements SortStrategy {
4     public void sort(int[] data) { /* bubble sort logic */ }
5 }
6 class QuickSort implements SortStrategy {
7     public void sort(int[] data) { /* quicksort logic */ }
8 }
9
10 class Sorter {
11     private SortStrategy strategy;
12     Sorter(SortStrategy s) { this.strategy = s; }
13     void setStrategy(SortStrategy s) { this.strategy = s; } // swap at runtime!
14     void sort(int[] data) { strategy.sort(data); }
15 }
16
17 Sorter s = new Sorter(new BubbleSort());
18 s.sort(arr);
19 s.setStrategy(new QuickSort()); // swap strategy at runtime
20 s.sort(arr);

```

Strategy vs Template Method:

	Strategy	Template Method
How	Composition (uses interface)	Inheritance (extends class)
Swap	At runtime	At compile time
Algorithm	Completely replaced	Steps can be overridden

Exam Tip

Strategy = plug-in behaviour via composition.

Template Method = fixed skeleton with customisable steps via inheritance.

6. SOLID Principles

Definition

SOLID is a set of five object-oriented design principles that make software more maintainable, flexible, and understandable.

S — Single Responsibility Principle (SRP)

A class should have only ONE reason to change.

Each class does exactly one thing.

```

1 // VIOLATION: Report class handles both data AND printing
2 class Report {
3     String generateReport() { ... }
4     void printReport() { ... } // two responsibilities!
5 }
6

```

```

7 // CORRECT:
8 class Report { String generateReport() { ... } }
9 class ReportPrinter { void print(Report r) { ... } }

```

O — Open/Closed Principle (OCP)

Open for extension, closed for modification.

Add new behaviour by adding new code, not by changing existing code.

```

1 // VIOLATION: Adding new shape requires modifying existing class
2 class AreaCalc {
3     double calc(Object shape) {
4         if (shape instanceof Circle) { ... }
5         else if (shape instanceof Square) { ... } // must modify here every time!
6     }
7 }
8
9 // CORRECT: Use abstraction
10 interface Shape { double area(); }
11 class Circle implements Shape { public double area() { return Math.PI * r * r; } }
12 class Square implements Shape { public double area() { return side * side; } }
13 class AreaCalc { double calc(Shape s) { return s.area(); } } // never needs
    modification!

```

L — Liskov Substitution Principle (LSP)

Subtypes must be substitutable for their base types without altering program correctness.

If S extends T, anywhere T is used, S must work correctly.

```

1 // VIOLATION: Square extends Rectangle but breaks its behaviour
2 class Rectangle { int w, h; void setW(int w){this.w=w;} void setH(int h){this.h=h;} }
3 class Square extends Rectangle {
4     void setW(int w) { this.w = this.h = w; } // forces h==w, violates Rectangle
        contract!
5 }
6
7 // CORRECT: Don't force the inheritance; use a common interface
8 interface Shape { int area(); }
9 class Rectangle implements Shape { ... }
10 class Square implements Shape { ... }

```

I — Interface Segregation Principle (ISP)

No client should be forced to depend on methods it doesn't use.

Prefer many small, specific interfaces over one large (fat) interface.

```

1 // VIOLATION: Fat interface
2 interface Worker { void work(); void eat(); void sleep(); }
3 class Robot implements Worker {
4     public void work() { ... }
5     public void eat() { /* robots don't eat! Forced to implement */ }
6     public void sleep() { /* robots don't sleep! */ }
7 }
8
9 // CORRECT: Split the interface
10 interface Workable { void work(); }
11 interface Eatable { void eat(); }
12 class Robot implements Workable { public void work() { ... } }
13 class Human implements Workable, Eatable { public void work(){...} public void eat()
    {...} }

```

D — Dependency Inversion Principle (DIP)

Depend on abstractions, not concretions.

High-level modules should not depend on low-level modules; both should depend on interfaces.

```
1 // VIOLATION: High-level class depends directly on low-level concrete class
2 class ReportService {
3     private MySQLDatabase db = new MySQLDatabase(); // tightly coupled!
4     void generate() { db.query("..."); }
5 }
6
7 // CORRECT: Both depend on an interface
8 interface Database { String query(String sql); }
9 class MySQLDatabase implements Database { ... }
10 class ReportService {
11     private Database db; // depends on abstraction
12     ReportService(Database db) { this.db = db; } // injected!
13     void generate() { db.query("..."); }
14 }
15 // Can now swap MySQL for PostgreSQL without touching ReportService
```

Exam Tip

SOLID Quick Memory Aid:

- **S** — One class, one job
- **O** — Add, don't edit
- **L** — Child replaces parent safely
- **I** — Small focused interfaces
- **D** — Code to interfaces, not classes

7. Software Development Life Cycle (SDLC)

7.1 SDLC Overview

Definition

The **SDLC** is a structured process followed to plan, design, develop, test, and deploy software. It provides a systematic approach that improves quality and predictability.

Phases:

1. **Planning** — feasibility, scope, resources, timeline
2. **Requirements Analysis** — functional and non-functional requirements
3. **System Design** — HLD (architecture) and LLD (detailed modules)
4. **Implementation** — actual coding
5. **Testing** — unit, integration, system, acceptance testing
6. **Deployment** — release to production
7. **Maintenance** — post-release support and enhancements

Importance: Reduces risk, improves collaboration, ensures quality, provides documentation.

7.2 SDLC Models

Waterfall Model

Waterfall

A **linear, sequential** model where each phase must be completed before the next begins. No going back.

Flow: Requirements → Design → Implementation → Testing → Deployment → Maintenance

Advantages	Disadvantages
Simple and easy to manage	No flexibility for changes
Clear milestones and documentation	Testing happens very late
Works well for fixed requirements	Client sees product only at the end
Easy to estimate timeline	Poor for complex or evolving projects

Best for: Projects with well-defined, stable requirements (e.g., government contracts, safety-critical systems).

Agile Model

Agile

An **iterative and incremental** approach where software is developed in short cycles called **sprints** (1–4 weeks). Welcomes changing requirements, continuous feedback, and collaboration.

Agile Manifesto values:

- Individuals and interactions **over** processes and tools
- Working software **over** comprehensive documentation
- Customer collaboration **over** contract negotiation
- Responding to change **over** following a plan

Scrum (popular Agile framework):

- **Sprint** — 1–4 week development cycle
- **Product Backlog** — prioritised list of features
- **Sprint Review** — demo working software to stakeholders
- **Retrospective** — team reflects on the process
- **Daily Stand-up** — 15-min daily sync meeting

Advantages	Disadvantages
Flexible — adapts to change	Harder to estimate final cost/timeline
Working software delivered early	Requires active client involvement
Continuous feedback reduces risk	Documentation can suffer
Better for complex/unclear requirements	Scope creep is common

Exam Tip

Waterfall vs Agile:

- **Waterfall:** sequential, fixed requirements, document-heavy, no going back
- **Agile:** iterative, evolving requirements, working software first, continuous delivery

8. Testing

8.1 What is Testing?

Definition

Software Testing is the process of evaluating a software system to detect differences between expected and actual behaviour (i.e., find **bugs**) and verify that the system meets requirements.

Goals: Detect defects early, validate requirements, ensure quality, build confidence in the system.

Verification vs Validation:

- **Verification** — “Are we building the product *right*?” (does it match design?)
- **Validation** — “Are we building the *right* product?” (does it meet user needs?)

8.2 Types of Testing

Testing by Scope

Type	What it tests	Who writes it
Unit Testing	Individual functions/methods in isolation	Developer
Integration Testing	Interaction between multiple units/modules	Developer / QA
End-to-End (E2E)	Entire system flow from user perspective	QA
Acceptance Testing	System meets business requirements	Client / QA

Unit Testing example (JUnit):

```

1 @Test
2 public void testAdd() {
3     Calculator c = new Calculator();
4     assertEquals(5, c.add(2, 3)); // expected=5, actual=c.add(2,3)
5 }

```

Testing Pyramid:

Acceptance / E2E (few, slow, expensive)
 Integration Tests (some)
 Unit Tests (many, fast, cheap)

Box Testing

Type	Tester knowledge	Focus
White Box	Full access to source code	Code paths, branches, coverage
Black Box	No access to internals	Input-output only (specs-based)
Grey Box	Partial knowledge	Mix of both approaches

Key Point

White box = structural testing. Checks *how* it works internally.
Black box = behavioural testing. Checks *what* it does from outside.

Manual vs Automated Testing

	Manual Testing	Automated Testing
Execution	Human tester	Scripts / tools (Selenium, JUnit)
Speed	Slow	Fast (can run thousands of tests)
Cost	High long-term	High setup, low ongoing
Best for	Exploratory, UX, ad-hoc	Regression, unit, repeated tests

Functional vs Non-Functional Testing

	Functional Testing	Non-Functional Testing
Verifies	What the system <i>does</i>	How well the system performs
Examples	Login, checkout, CRUD operations	Performance, security, load testing
Based on	Functional requirements	Quality attributes / NFRs

Exam Tip

Common MCQ tricks:

- “Testing without seeing code” → Black Box
- “Testing a single method” → Unit Test
- “User logs in and places an order, all steps tested together” → E2E
- “Client verifies requirements are met” → Acceptance Test
- “Performance / Load / Stress testing” → Non-Functional
- “Regression testing runs automatically on every commit” → Automated

Quick Reference Cheat Sheet

OOP Pillars

- Encapsulation — hide data, control access
- Abstraction — hide implementation, show interface
- Inheritance — IS-A, reuse via extends
- Polymorphism — many forms, overload/override

Design Patterns

- **Singleton** — one instance, global access
- **Factory** — centralise object creation
- **Adapter** — wrap incompatible interface
- **Composite** — tree of objects, part-whole
- **Observer** — notify all subscribers
- **Strategy** — swap algorithms at runtime

SOLID

- S — one responsibility
- O — extend, don't modify
- L — substitutable subtype
- I — small interfaces
- D — depend on abstractions

ERD Cardinalities

- 1:1 — Person : Passport
- 1:N — Teacher : Students
- M:N — Students : Courses (needs join table)

UML Relationships

- Solid arrow = Association
- Hollow diamond = Aggregation
- Filled diamond = Composition
- Hollow triangle = Inheritance
- Dashed + hollow = Realisation

Testing Scope

- Unit → Integration → E2E → Acceptance
- White (code visible) vs Black (code hidden)
- Functional (what) vs Non-functional (how well)

SDLC Models

- Waterfall — linear, fixed reqs, doc-heavy
- Agile — iterative sprints, flexible reqs